DRUMSCRIBE AI

REVISED PRODUCT REQUIREMENTS DOCUMENT

Version 2.0 · Engineering execution specification

Primary deliverable
A complete, editable, professionally engraved drum-set score PDF generated from a full mixed music recording.

Development priority: Accuracy → Reliability → Musical completeness → Usability → Visual polish

Document contents

- 1. Executive summary and scope
- 2. Deliverables and development environment
- 3. Model selection, allow-list, and licensing
- 4. Test data and benchmark requirements
- 5. Audio detection and rhythm interpretation
- 6. Drum classification and source separation
- 7. Notation, PDF generation, and technology stack
- 8. Application structure and MVP acceptance test
- 9. Implementation roadmap and session management
- 10. CLAUDE.md, setup scripts, feature status, and release rules
- 11. Claude's immediate instructions and definition of success

1. Executive summary

DrumScribe AI analyzes a complete mixed music recording and reconstructs its drum performance as detailed, professional drum-set sheet music. It must identify kick and snare hits, ghost notes, subdivisions, hi-hat articulations, cymbals, toms, accents, dynamics, fills, rudiments where identifiable, swing, shuffle, tuplets, tempo changes, meter changes, simultaneous events, and musical transitions.

The output should approach the completeness of professional drum education repertoire, including Rockschool-style educational transcription standards, without copying copyrighted arrangements or proprietary layouts.

Core engineering philosophy: distinguish what the audio contains, what models detect, how events are interpreted rhythmically, and how they are represented in notation. A detected transient is not automatically a drum hit; a drum hit is not automatically a ghost note; an onset sequence is not proof of one tempo or time signature.

2. Product scope

2.1 Primary input

The main input is a complete stereo music recording containing drums and other instruments: lawful local mixed recordings, full-band rehearsals, studio recordings, live recordings, compressed or uncompressed audio. The application must not require an isolated drum stem.

2.2 Secondary inputs

- Isolated drum recordings and separated stems.
- Synthetic drum recordings with known MIDI events.
- Reference MIDI performances.
- User-provided tempo and time-signature hints.

2.3 Out of scope for first release

- Guaranteed perfect transcription from every recording.
- Automatic identification of the physical drummer.
- Guaranteed recovery of actual hand sticking from audio alone.
- Bypassing Spotify, YouTube, or other platform restrictions.
- Training a large foundation audio model from scratch.
- Real-time transcription during live performance.

Spotify and YouTube support may be added through authorized integrations. Local audio upload is mandatory for the first release.

3. Product deliverables

Deliverable	Requirement
PDF	Complete, professionally engraved drum score
MusicXML	Editable standard notation
MIDI	Reconstructed drum performance
JSON	Timestamped events, labels, confidence, and timing hypotheses
Audio analysis	Drum stem and relevant intermediate outputs where available
Report	Tempo, meter, detected instruments, confidence, and validation results

The PDF must cover the complete recording, including its ending. It must not silently omit difficult measures, fast fills, quiet passages, or uncertain events.

4. Development environment requirements

4.1 Required environment inventory

At the start of every session, inspect and report OS/version, CPU/cores, RAM, disk space, GPU/CUDA availability, Python, PyTorch, audio libraries, MuseScore/version, headless system libraries, Docker, Git, network connectivity, and storage for weights/intermediate audio. Create ENVIRONMENT_REPORT.md. Do not assume a GPU exists.

4.2 CPU-first requirement

- Provide a viable CPU-only development and execution path.

- Do not repeatedly attempt CUDA installation when no GPU exists.
- Use compatible CPU inference and smaller models where practical.
- Process audio in manageable chunks and explain performance trade-offs.
- Keep the architecture GPU-capable without requiring a rewrite.

4.3 Dependency installation policy

- Check existing installations and version compatibility first.
- Estimate download size, disk needs, host reachability, and execution time.
- Use a prebuilt compatible environment where available.
- Split large installs into resumable steps.
- Do not restart a failed large installation without diagnosing the cause.

4.4 Required dependency hosts

Check access, where relevant, to PyPI, the required PyTorch package index, Hugging Face, GitHub repositories/releases, MuseScore distribution/dependencies, and model-specific weight hosts. Record failures as environment limitations, not application bugs.

4.5 Prebuilt environment

Prefer a reproducible environment containing compatible Python, CPU-compatible PyTorch, audio dependencies, approved model dependencies, MuseScore CLI, required headless libraries, and test/lint tools.

- requirements.txt or pyproject.toml; requirements-lock.txt.
- setup.sh and setup.ps1; environment.yml if Conda is used.
- Dockerfile only if Docker is available and tested.
- ENVIRONMENT_REPORT.md.

An unbuilt or unexecuted Dockerfile must be marked UNVERIFIED.

4.6 Model-weight preparation

Download model weights once, verify and cache them. Track model name/version, source, license, weight location, checksum, dependencies, hardware requirements, download status, and execution status. Do not silently substitute another model.

5. Model selection and allow-list

Candidates must be evaluated for this task. These are not claims that they are installed or approved for production.

Component	Candidate	Usage
Drum separation	Demucs-family models	Optional drum-stem extraction
Drum transcription	ADTOF	Candidate event/transcription system
Feature extraction	librosa	Signal analysis and musical features
Audio processing	NumPy + SciPy	Signal processing
ML framework	PyTorch	Compatible model inference
Notation	MusicXML	Intermediate notation
Engraving	MuseScore CLI	PDF generation

5.1 ADTOF evaluation

- Inspect official repository and license.
- Identify Python/PyTorch versions, architecture, and pretrained weights.
- Verify whether it supports full mixed recordings or requires isolated drums.
- Determine supported instruments and articulations.
- Run an authorized test against known annotations.
- Do not claim integration until it has actually executed.

5.2 Demucs evaluation

Demucs is optional for the MVP. Integrate only if code and weights are obtainable, license-compatible, dependencies install, execution succeeds, and benchmark results show improvement. The application must continue without it.

5.3 Production model approval

Every model must have a MODEL_REGISTRY.md entry. Production approval requires license review, dependency review, weight verification, execution test, benchmark evaluation, and documented limitations.

6. Licensing and commercial use

Keep the architecture suitable for eventual commercial distribution. Prefer commercially compatible dependencies. Verify repository, weight, and dataset licenses independently. Public GitHub availability does not imply commercial rights.

- Do not use non-commercial models or datasets in a commercial release without permission.
- Record attribution and license obligations; maintain a software bill of materials.
- Label restricted models NON-COMMERCIAL — NOT APPROVED FOR PRODUCTION.
- Treat the initial project as a development prototype, not an automatically authorized commercial product.

Commercial release requires a separate license audit.

7. Test data and benchmark requirements

Do not claim real-world accuracy based exclusively on synthetic drum recordings.

7.1 Reference datasets

- ENST Drums: investigate recordings, annotations, access terms, and task suitability.
- IDMT-SMT-Drums: investigate recordings, labels, instrument categories, and license.
- Slakh2100: investigate multitrack/MIDI material for controlled mixture testing; do not assume rendered MIDI perfectly describes human performance or every final-mix detail.
- Investigate additional authorized datasets and document actual contents and license.

7.2 Required reference clips

Create 3–5 licensed clips before claiming full-mix accuracy:

- Straightforward rock/pop full mix.
- Funk recording with quiet snare ghost notes.
- Fast fills, toms, and cymbal changes.
- Swing or shuffle recording.
- Dense mix with overlapping instruments and rapid kick/snare patterns.

Each clip needs audio, sample rate, channels, duration, known tempo/meter, ground-truth event timestamps, instrument labels, articulation labels where known, annotation source, and usage license. If unavailable, mark the real-world benchmark BLOCKED. Do not fabricate annotations.

7.3 Benchmark categories

Benchmark	Purpose
A. Synthetic controlled tests	Exact onsets, simultaneous hits, sample-zero events, known patterns
B. Isolated drum recordings	Instrument/articulation classification under favorable conditions
C. Full mixed recordings	Actual product performance

Never combine these categories into one misleading accuracy figure.

7.4 Initial numeric targets

Metric	Initial target
Kick event F1	≥ 0.80
Snare event F1	≥ 0.80
Kick/snare onset tolerance	±50 ms
Hi-hat event F1	≥ 0.70
Tom event F1	≥ 0.70
Ghost-note F1	≥ 0.60
Synthetic-reference drum recall	≥ 0.95
Sample-zero event detection	100%
MusicXML structural validation	100%
PDF generation on supported projects	100%

These are targets, not achieved results. Report precision, recall, F1, reference event counts, false positives, median and 95th-percentile onset error. Evaluate timing at ±10, ±25, ±50, and ±100 ms. A target is achieved only on a suitable held-out set.

8. Audio detection architecture

Pipeline: audio decoding → signal validation → original-mix analysis → optional source separation → candidate onset detection → instrument classification → articulation classification → rhythm inference → musical structure reconstruction → event validation → notation generation.

8.1 Full-mix detection

Full-band recordings contain percussive guitar attacks, bass transients, distorted chords, and cymbal-like noise. Do not classify every transient as a drum. Require corroborating frequency, transient-envelope, temporal-context, separation, confidence, and instrument-class evidence. Preserve uncertain candidates rather than forcing them into the score.

8.2 Simultaneous events

Support multiple instruments at the same timestamp: kick and snare, kick and crash, snare and hi-hat, multiple toms. Use multi-label events, not a single mutually exclusive class per timestamp.

8.3 First-sample event handling

Test events at sample 0, sample 1, analysis-window boundaries, chunk boundaries, and simultaneous events at the beginning. Avoid algorithms that discard the first frame. Add regression tests.

9. Rhythm and musical interpretation

9.1 Ambiguous tempo

Onset patterns may support half-time, double-time, or other metrical interpretations. Generate candidate tempo hypotheses, compare beat phase and downbeat evidence, use bass/harmonic context where available, consider section consistency, and preserve alternatives when unresolved.

9.2 Ambiguous time signatures

Some passages may reasonably be interpreted as 2/4 or 4/4 depending on phrase and beat interpretation. Display alternatives, explain ambiguity, avoid silently selecting a definitive meter, and allow user choice.

9.3 Swing and shuffle

Do not infer straight subdivisions solely from onset spacing. Evaluate beat-phase consistency, swing ratio, local subdivision timing, accent patterns, and phrase-level consistency.

9.4 Tuplet protection

Prevent overfitting noisy events with five- and six-note tuplets. Use complexity penalties, minimum confidence, repeated-structure evidence, simpler-grid comparisons, musical-context checks, and user-visible alternatives. Do not introduce unusual tuplets without evidence.

10. Drum event classification

Prioritize kick, snare, closed hi-hat, open hi-hat, ride, crash, and toms in the first release. Advanced classes, where supported: ghost notes, accents, flams, drags, rolls, double strokes, rimshots, cross-sticks, hi-hat pedal chick, partial hi-hat opening, cymbal bell/choke, splash/China, double-kick patterns, and buzz strokes.

Every event must preserve event ID, raw timestamp, sample position, instrument label/confidence, articulation label/confidence, estimated velocity, beat/bar hypothesis, subdivision hypothesis, ghost-note probability, original model output, user-edited status, and provenance. Raw timestamps must never be overwritten by quantization.

11. Source separation

Source separation is optional for the MVP and required only for a later advanced milestone. If weights are unavailable, do not repeatedly retry blocked downloads. Check verified cached weights, continue with original-mix detection, and report the limitation.

Compare separated and original audio on kick recall, snare recall, ghost-note recall, hi-hat classification, and false-positive rate. Do not assume separation always improves accuracy.

12. Notation and PDF generation

Selected architecture: Internal event representation → Rhythmic reconstruction → MusicXML → MuseScore CLI → PDF.

12.1 MuseScore environment

- Verify installation and version.
- Verify command-line interface and required system libraries.
- Test headless execution and PDF output.
- Verify page count and output integrity.
- Inspect a rendered test score.

Do not mark PDF generation complete until a real PDF has been rendered and inspected.

12.2 Notation requirements

- Percussion clef and configurable drum staff mapping.
- Kick/snare noteheads, hi-hat and cymbal markings, toms.
- Ghost notes and parentheses, accents, grace notes, flams and drags.
- Triplets, tuplets, ties, rests, correct beams, dynamics.
- Rehearsal marks, repeat signs, tempo/meter changes, suggested sticking.
- Full-song page layout.

12.3 PDF validation

- Full recording coverage and correct measure durations.
- Valid notes, rests, simultaneous events, tuplets.
- No missing glyphs, clipped notes, or broken page layouts.
- Readable pages and successful PDF opening.

Render pages to images for visual inspection where possible.

12.4 Playhead synchronization

Later editor milestone: current-measure highlighting, playhead synchronization, click-to-seek, page navigation, zoom, and horizontal scrolling. Do not mark complete until tested on a multi-page score.

13. Selected technology stack

Layer	Selected technology
Frontend	React + TypeScript
Backend	Python + FastAPI
Audio processing	NumPy + SciPy + librosa
ML framework	PyTorch
Model integration	ADTOF, subject to validation and license review
Source separation	Demucs, optional and subject to validation
Internal event format	Versioned JSON schema
Notation	MusicXML
Engraving	MuseScore CLI
Testing	pytest
Frontend testing	Vitest
Linting	Ruff + TypeScript linting
Version control	Git

Pin compatible versions. Verify installed versions and API signatures, write executable smoke tests, and add regression tests. Update code and lockfiles together when APIs change.

14. Application structure

Proposed repository layout:

```
drumscribe-ai/
■■■ CLAUDE.md · README.md · PROGRESS.md
■■■ ENVIRONMENT_REPORT.md · MODEL_REGISTRY.md
■■■ LICENSE_AUDIT.md · setup.sh · setup.ps1
■■■ pyproject.toml · requirements-lock.txt
■■■ backend/{api,audio,separation,detection}
■ ■■■ {rhythm,reconstruction,notation,engraving,validation,tests}
■■■ frontend/{src,components,player,score-viewer,editor}
■■■ models/model-manifest.json
■■■ data/{incoming,processed,stems,benchmarks,exports}
■■■ scripts/{check_environment,download_models,validate_models}
■ ■■■ {run_benchmarks,validate_pdf}
■■■ docs/ · outputs/{pdf,midi,musicxml}
```

Do not create empty modules just to make the directory look complete. Implement modules as their milestones begin.

15. Single MVP acceptance test

Test: Full-mix audio to complete drum score. Input: one authorized stereo WAV or MP3 containing a full musical mix, not an isolated drum track; approximately 60–120 seconds; corresponding ground-truth drum-event annotations. Environment: CPU-compatible, no GPU requirement.

Execution: Import; decode and validate; analyze original mix; detect candidate events; classify kick/snare; preserve simultaneous events; estimate tempo and beat positions; provide alternatives if tempo/meter is ambiguous; construct full event timeline; generate MusicXML; render with MuseScore CLI; validate PDF; export MIDI and JSON.

Acceptance conditions:

- Entire recording represented.
- Kick and snare F1 each ≥ 0.80 at ±50 ms on held-out acceptance clip.
- Sample-zero events retained; simultaneous kick/snare represented separately.
- No unsupported complex tuplets introduced to fit noise.
- MusicXML passes structural validation.
- PDF opens, renders, and covers the entire recording.
- JSON retains original timestamps and confidence.
- Benchmark report includes precision, recall, F1, event count, and timing errors.

If any condition fails, MVP is not accepted. Synthetic-only performance cannot substitute. If the licensed full-mix clip and answer key are unavailable, the test is BLOCKED; development testing may continue, but real-world acceptance cannot be claimed.

16. Implementation roadmap

Milestone	Deliverables and exit criteria
M0 — Environment	Environment report; dependency manifest; setup script; model registry; license audit; MuseScore headless test; audio decode test; model availability. Exit: reproducible CPU environment and documented limitations.
M1 — MVP	Full-mix input; preprocessing; onset detection; kick/snare baseline; simultaneous events; raw timestamps; initial beat/tempo; MusicXML; PDF; MIDI/JSON; acceptance test. Exit: single MVP test passes.
M2 — Detection/separation	Validated pretrained model; optional separation; original-vs-separated comparison; improved classification; confidence; duplicate suppression; false-positive analysis. Exit: measured improvement.
M3 — Rhythm	Beat/downbeat; swing/shuffle; adaptive quantization; tempo/meter alternatives; tuplet protection; measure reconstruction; section markers. Exit: regression tests pass.
M4 — Articulations	Ghost notes; hi-hat behavior; pedal events; flams/drags/rolls; cymbals; tom fills; double-kick; dynamics. Exit: separate benchmark per supported articulation.
M5 — Editor	Audio player; synchronized score; playhead; measure navigation; correction; confidence; undo/redo; score regeneration. Exit: edits update score/export; multi-page sync verified.
M6 — Hardening	Long-track processing; resource management; privacy; recovery; packaging; docs; reproducible installation; regression suite. Exit: clean setup and end-to-end regression.

17. Development process and session management

17.1 Git discipline

- Initialize Git immediately; commit after each completed milestone.
- Use meaningful commit messages.
- Do not commit model weights or large generated audio.
- Maintain .gitignore and keep secrets out of source control.
- Do not mark a milestone complete before tests pass.

17.2 Session checkpoints

At every session end, update PROGRESS.md with current milestone, completed/verified/unverified/blocked tasks, current Git commit, commands executed, test results, and next three actions. The next session must read CLAUDE.md, PROGRESS.md, and ENVIRONMENT_REPORT.md.

17.3 Long-running jobs

Split large installs, model downloads, audio analysis, benchmarks, and PDF rendering into manageable resumable operations. Capture logs and exit codes. Starting a background process is not success.

17.4 Permission handling

- A chat approval is not an operating-system permission grant.
- Identify the exact blocked command and missing permission.
- Explain minimum safe permission; ask through the actual permission mechanism.
- Wait for authorization and verify execution.
- Never repeatedly attempt the same blocked operation or claim unexecuted code ran.

17.5 Stop-and-ask rule

Stop and ask when permissions are unavailable, model licensing is incompatible, test data is missing, dependencies cannot be installed through an approved route, model assets cannot be verified, architecture scope must change, or required access is unavailable. Do not silently make high-impact assumptions.

18. Required CLAUDE.md

Create CLAUDE.md at the repository root as the operating manual for every Claude development session.

- Project objective: full-mix drum transcription to complete, validated notation/PDF.
- Read PROGRESS.md and ENVIRONMENT_REPORT.md before work/installations.

- Do not assume GPU availability or redownload verified weights.
- Do not install huge dependencies without checking limits.
- Never claim untested functionality works or fabricate benchmark results.
- Never substitute synthetic accuracy for real-mix accuracy.
- Preserve raw timestamps; report ambiguous tempo/meter as alternatives.
- Stop and ask when blocked; commit after each milestone.

18.1 Required command categories

```
python scripts/check_environment.py
pytest backend/tests -q
python scripts/run_benchmarks.py
python scripts/validate_pdf.py
cd frontend && npm test
cd frontend && npm run lint
ruff check backend
uvicorn backend.api.main:app --reload
```

These are expected command categories. Include them only after implemented and tested; update actual entry points to match the repository. Never present placeholders as working commands.

18.2 Completion report

Every milestone report must state Implemented, Tested, Verified, Unverified, Blocked, and Known limitations. The words complete, working, and accurate must be supported by test evidence.

19. Setup script requirements

Provide setup.sh and setup.ps1. Check OS, Python, disk, existing dependencies, virtual environment, pinned dependencies, CPU PyTorch availability, MuseScore, system libraries, model cache, checksums, and a small smoke test. Print success/warning/failure. Do not download multi-gigabyte models without checking network, storage, and user approval. Scripts must be safe to rerun. If one dependency fails, report it and continue unrelated safe setup tasks.

20. Feature status classification

Status	Meaning
NOT STARTED	No implementation exists
IMPLEMENTED	Code exists
TESTED	Relevant tests have run
VERIFIED	Tests passed in intended environment
EXPERIMENTAL	Works in limited conditions; known limitations
BLOCKED	Cannot proceed due to environment/external constraint
UNVERIFIED	Code exists but has not been adequately tested

Never use 'completed' as a substitute for 'verified'.

21. Final acceptance and release rules

The product may be called an MVP only after M1 passes its acceptance test. It may be called an advanced transcription system only after full-mix detection is benchmarked; ghost notes and advanced articulations are separately evaluated; rhythm is tested on swing and ambiguous meter; exports are validated; multi-page navigation works; environment is reproducible; and licensing is audited.

Commercial release additionally requires commercial license clearance for every bundled model, weight file, dataset-derived component, and dependency.

22. Claude's immediate instructions

- **Step 1 — Audit:** Inspect repository, environment, code, dependencies, cached models, permissions, and tests.
- **Step 2 — Report:** Create ENVIRONMENT_REPORT.md, MODEL_REGISTRY.md, LICENSE_AUDIT.md, and PROGRESS.md.

• **Step 3 — Feasibility:** Verify audio decoding, CPU PyTorch if needed, MuseScore headless rendering, model availability, and package/host access.

• **Step 4 — Data:** Identify benchmark datasets and test-data specification. If full-mix references and answer keys are missing, mark acceptance test BLOCKED.

• **Step 5 — Build M1:** Implement the smallest complete audio-to-event-to-MusicXML-to-PDF pipeline.

• **Step 6 — Test:** Run the single MVP acceptance test and report actual metrics.

• **Step 7 — Improve:** Only after M1 is stable, begin separation, advanced articulation detection, and additional models.

• **Step 8 — Checkpoint:** Commit milestone, update PROGRESS.md, document next steps.

Do not redesign the frontend first, implement every advanced feature at once, install every possible model, or create empty modules. If blocked, stop and ask for the specific resource or permission.

23. Definition of success

DrumScribe AI succeeds when a drummer can upload an ordinary full-band recording and receive a complete, editable drum score that captures the actual performance with measurable accuracy.

• Ghost notes and accents.

• Fills and subdivisions.

• Cymbal articulations.

• Simultaneous hits.

• Dynamic variations and musical transitions.

• Tempo and meter interpretation.

The software must be honest about ambiguous or inaudible details. A professionally engraved but musically inaccurate score is a failed transcription. A sophisticated model that cannot execute in the available environment is not an implemented feature. A synthetic benchmark that does not generalize to mixed recordings is not proof of product accuracy.

Build a smaller, measurable, functioning transcription system first. Then progressively make it more musically intelligent.